CHAPTER 7

CONTROL STATEMENTS

CHAPTER ESSENCE AT A GLANCE:
Control flow constructs alter linear program execution through conditional selection branching and iterative loop sequencing to execute decision logic.

4 CORE PRINCIPLES TO UNDERSTAND FIRST

• **Selection Branching Structures:** Conditional branching evaluates boolean tests to direct execution flow down mutually exclusive paths using if, if-else, and cascaded else-if ladders.

• **Multi-Way Dispatch and Fall-Through:** The switch construct provides efficient jump-table dispatch for discrete integral expressions, requiring explicit break statements to prevent unintentional fall-through into succeeding cases.

• **Loop Topologies and Invariants:** Iteration constructs automate repetitive calculations; entry-controlled loops (for, while) test conditions prior to execution, whereas exit-controlled loops (do-while) evaluate after body execution.

• **Jump Redirection Statements:** Jump statements immediately alter flow; break terminates the enclosing loop or switch construct, continue skips to the next iteration cycle, and return exits the current function.

ESSENTIAL ARCHITECTURE & CORE SYNTAX CARDS

$$\text{Selection Branch} = \textbf{if}\ (\text{Cond})\ \{\text{True Block}\}\ \textbf{else}\ \{\text{False Block}\}$$

Two-Way Decision Branching Mechanism

$$\textbf{switch}(\text{Integral Expr}) \rightarrow \textbf{case}\ c_1 \rightarrow \ \dots\ \rightarrow \textbf{case}\ c_k \rightarrow \textbf{default}$$

Discrete Multi-Way Dispatch Switch Model

$$\text{Loop Typology} = \frac{\text{Entry-Controlled: } \textbf{for, while} \quad (\text{Test condition at loop entry})}{\text{Exit-Controlled: } \textbf{do-while} \quad (\text{Test at exit, executes } \geq 1 \text{ time})}$$

Entry-Controlled vs Exit-Controlled Looping

$$\textbf{break} \rightarrow \text{Terminates Loop / Switch} \quad | \quad \textbf{continue} \rightarrow \text{Skips to Next Iteration}$$

Jump Statement Flow Redirection Operators

CRUCIAL EXAM PRECAUTIONS & VALUATION RUBRIC SECRETS

• **Alert:** Omitting the break statement in a switch construct causes automatic fall-through, executing all following cases until a break or the end of the block is reached.

• **Alert:** A while loop tests its condition at entry and may execute zero times, whereas a do-while loop always executes its body at least once.

• **Alert:** Do not place a semicolon directly after if(condition) or for(...); doing so terminates the statement prematurely, making the body execute unconditionally.

HIGH-YIELD CENTUM QUESTIONS & STEP-BY-STEP EXPLANATIONS

1. Distinguish between entry-controlled loops and exit-controlled loops in C++. [Score: 2 Marks]

Answer & Key Valuation Steps:

• Entry-Controlled Loops: The loop control condition is tested before entering the loop body (e.g. for and while loops); if the condition is false initially, the body executes zero times. **[1 Mark]**

• Exit-Controlled Loops: The control condition is tested after executing the loop body (e.g. do-while loop); the loop body is guaranteed to execute at least once regardless of condition. **[1 Mark]**

2. What is the purpose of the `default` label in a `switch` statement? Is it mandatory? [Score: 2 Marks]

Answer & Key Valuation Steps:

• Purpose: The default label executes when the switch expression value fails to match any of the preceding case constant values, providing fallback logic. **[1 Mark]**

• Optional Nature: It is completely optional; if omitted and no case matches, the switch statement simply finishes execution without doing anything. **[1 Mark]**

3. Compare `switch-case` statement with `else-if` ladder, highlighting their limitations. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Expression Types: switch evaluates strictly integral expressions (int or char); else-if ladder evaluates complex floating-point and boolean relational conditions. **[1 Mark]**

• Range Testing: switch tests only for exact equality against discrete constant values; else-if ladder can test continuous numeric ranges (e.g. marks >= 80). **[1 Mark]**

• Clarity: switch provides cleaner structure for multi-way menu choices; else-if ladder provides greater algorithmic flexibility. **[1 Mark]**

4. Differentiate between `break` and `continue` statements with code snippets. [Score: 3 Marks]

Answer & Key Valuation Steps:

• break Statement: Causes immediate termination of the enclosing loop or switch construct, transferring control to the statement following the loop. **[1.5 Marks]**

• continue Statement: Skips the remaining statements in the current loop iteration and immediately transfers control to the loop condition test for the next cycle. **[1.5 Marks]**

5. Write a C++ program using a `for` loop to compute the factorial of a positive integer N. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Program Setup: #include <iostream>\
using namespace std;\
int main() { int n; long long fact = 1; cout << "Enter N: "; cin >> n; **[1 Mark]**

• Loop Execution: for(int i = 1; i <= n; i++) { fact = fact * i; } **[1 Mark]**

• Display Result: cout << "Factorial of " << n << " = " << fact << endl; return 0; } **[1 Mark]**

6. Convert a given `while` loop into an equivalent `for` loop and `do-while` loop. [Score: 3 Marks]

Answer & Key Valuation Steps:

• Original while Loop: int i = 1; while(i <= 10) { cout << i << " "; i++; } **[1 Mark]**

• Equivalent for Loop: for(int i = 1; i <= 10; i++) { cout << i << " "; } **[1 Mark]**

• Equivalent do-while Loop: int i = 1; do { cout << i << " "; i++; } while(i <= 10); **[1 Mark]**

7. Write a C++ program to input a positive integer and determine whether it is a prime number or not. [Score: 4 Marks]

Answer & Key Valuation Steps:

• Setup and Input: #include <iostream>\
using namespace std;\
int main() { int num; bool isPrime = true; cout << "Enter positive integer: "; cin >> num; **[1 Mark]**

• Boundary Validation: if(num <= 1) { isPrime = false; } else { **[1 Mark]**

• Divisibility Test Loop: for(int i = 2; i * i <= num; i++) { if(num % i == 0) { isPrime = false; break; } } } **[1 Mark]**

• Output Result: if(isPrime) cout << num << " is a Prime Number." << endl; else cout << num << " is Not a Prime Number." << endl; return 0; } **[1 Mark]**